

Machine Learning Algorithms Handbook

Supervised, Unsupervised, and Scenario-Based Selection Guide

A practical reference for model choice, internals, and project flow

Purpose: give you a compact but detailed map of the main ML algorithms you are expected to know, how they work internally, and when to use them in real projects.

Scope: classical supervised learning, major unsupervised learning methods, and scenario-based recommendations for tabular, text, anomaly, segmentation, and low-latency systems.

Note: this is a high-value foundation guide, not a math encyclopedia. It focuses on the algorithms that show up repeatedly in interviews and production work.

1. Basic ML Project Flow

1. Define the business problem, the prediction target, and the error cost.
2. Collect data, label it, and inspect for leakage, imbalance, duplicates, and missing values.
3. Split data before fitting any preprocessing step that learns from data.
4. Build preprocessing pipelines for scaling, encoding, and imputation.
5. Start with simple baselines, then move to stronger models.
6. Tune using the correct validation strategy: random CV for i.i.d. data, time-aware splits for time series, group splits when entities repeat.
7. Evaluate using both model metrics and business metrics.
8. Package inference logic with the exact same preprocessing used in training.
9. Monitor drift, latency, and performance after deployment.

What to do before scaling or normalization

Always split first, then fit the scaler only on the training set. After that, transform train, validation, and test using the fitted scaler. If you scale on the full dataset first, you leak information from the future or from the test set into training.

Typical order: train/test split -> fit imputer/scaler/encoder on train -> transform all splits -> train model -> validate -> test.

2. Supervised Learning Algorithms

Supervised learning uses labeled examples (x, y) . The algorithm learns a function that maps inputs to targets. The main families are regression, classification, and ranking.

Algorithm	Core idea	Best for	Main caveat
Linear Regression	Fits a linear relationship by minimizing squared error.	Continuous targets with near-linear structure.	Sensitive to outliers and non-linear patterns.
Logistic Regression	Estimates class probability using a linear score passed through sigmoid/softmax.	Binary or multiclass classification with strong baselines.	Needs good feature engineering for complex patterns.
k-NN	Predicts from the labels of nearest points in feature space.	Small to medium datasets, local similarity problems.	Slow inference, sensitive to scaling and noisy dimensions.
Naive Bayes	Applies Bayes rule with conditional independence assumption.	Text classification, spam detection, sparse count data.	Independence assumption is often false.
Decision Tree	Recursively splits data to reduce impurity or variance.	Interpretability and mixed feature types.	Overfits easily unless constrained.
Random Forest	Averaged ensemble of many bootstrapped decision trees.	Robust tabular classification/regression.	Less interpretable than a single tree.
Gradient Boosting	Builds trees sequentially to correct residual errors.	High-accuracy tabular problems.	Can overfit if not regularized.
SVM	Finds a maximum-margin boundary, optionally in kernel space.	High-dimensional sparse data, medium-sized datasets.	Kernel methods do not scale well.
MLP / Feedforward NN	Learns layered non-linear transformations.	Complex nonlinear interactions in structured data.	Needs careful tuning and more data.

How the main supervised algorithms work

Linear Regression

- Core mechanism: minimize the distance between predicted and actual continuous values using least squares or regularized variants like Ridge and Lasso.
- Why it works: if the relationship is close to linear, the model is easy to optimize, fast, and surprisingly strong as a baseline.
- Use when: you need forecasted quantity, pricing, demand, or risk score with a mostly linear trend.
- Watch out for: multicollinearity, outliers, and misspecified non-linear relationships.

Logistic Regression

- Core mechanism: learn a linear score and convert it into probability with sigmoid (binary) or softmax (multiclass).
- Why it works: it optimizes log-likelihood and gives calibrated probabilities better than many harder models.
- Use when: you need a transparent classifier for churn, fraud pre-screening, lead scoring, or medical triage.
- Watch out for: weak performance if the boundary is highly non-linear without feature engineering.

k-Nearest Neighbors

- Core mechanism: store the training set, compute distances at prediction time, and vote among nearest points.
- Why it works: local similarity can be powerful when nearby examples are genuinely similar.
- Use when: the dataset is small and the feature space is meaningful after scaling.
- Watch out for: expensive inference, curse of dimensionality, and extreme sensitivity to feature scaling.

Naive Bayes

- Core mechanism: apply Bayes theorem with an assumption that features are conditionally independent given the class.
- Why it works: for sparse text counts, the independence assumption is wrong but still often good enough to separate classes.
- Use when: spam filtering, document classification, intent routing, or fast baseline NLP.

- Watch out for: correlated features and probability estimates that can be poorly calibrated.

Decision Tree

- Core mechanism: split data step by step using a feature and threshold that best reduces impurity (classification) or variance (regression).
- Why it works: it learns nonlinear rules and feature interactions without explicit feature engineering.
- Use when: interpretability matters or as the base learner in ensembles.
- Watch out for: high variance and overfitting if the tree grows too deep.

Random Forest

- Core mechanism: train many trees on bootstrap samples and average their predictions.
- Why it works: bagging reduces variance because errors from individual trees cancel out.
- Use when: you want a strong, stable tabular model with limited tuning.
- Watch out for: slower inference and less elegant explanations than a single tree.

Gradient Boosting (XGBoost, LightGBM, CatBoost)

- Core mechanism: add trees sequentially, where each new tree focuses on the mistakes made by the current ensemble.
- Why it works: the ensemble keeps correcting its residual errors, which gives very strong performance on tabular data.
- Use when: accuracy on structured/tabular data matters most.
- Watch out for: overfitting, leakage in target encoding, and tuning complexity.

SVM

- Core mechanism: find a boundary with the maximum margin between classes; kernels allow non-linear separation by working in transformed space.
- Why it works: margin maximization is a strong inductive bias for generalization.
- Use when: high-dimensional sparse data, moderate dataset sizes, and good feature scaling.
- Watch out for: kernel scalability and sensitivity to hyperparameters and scaling.

Neural Networks / MLP

- Core mechanism: stack linear layers and non-linear activations so the model can learn complex feature interactions.
- Why it works: each layer learns a progressively richer representation.
- Use when: you have large data or feature interactions that simpler models cannot capture well.
- Watch out for: initialization, overfitting, unstable training, and the need for careful regularization.

3. Unsupervised Learning Algorithms

Unsupervised learning works without labels. The goal is to discover structure: clusters, latent factors, density, anomalies, or lower-dimensional representations.

Algorithm	Core idea	Best for	Main caveat
K-Means	Moves centroids to minimize within-cluster squared distance.	Well-separated spherical clusters.	Needs K and struggles with non-spherical shapes.
Hierarchical Clustering	Builds a tree of merges or splits between points/clusters.	Exploratory analysis and dendrograms.	Can be slow on large datasets.
DBSCAN	Finds dense connected regions and labels sparse points as noise.	Arbitrary-shaped clusters and outlier detection.	Hard to tune for varying density.
Gaussian Mixture Model	Assumes data comes from a mixture of Gaussian components, fitted by EM.	Soft clustering and probabilistic assignment.	Sensitive to initialization and covariance assumptions.
PCA	Projects data onto directions of maximum variance.	Compression, denoising, visualization, collinearity reduction.	Linear only; may miss nonlinear structure.
t-SNE	Preserves local neighborhood structure in low-dimensional embedding.	Visualization of complex manifolds.	Not for general-purpose feature engineering.
UMAP	Builds a graph-based low-dimensional embedding preserving local and some global structure.	Fast visualization and manifold learning.	Sensitive to hyperparameters and interpretation.
Isolation Forest	Isolates anomalies by random splits; anomalies need fewer splits.	Anomaly detection on tabular data.	Not a clustering method.

How the main unsupervised algorithms work

K-Means

- Core mechanism: assign points to the nearest centroid, then recompute centroids as the mean of each cluster, repeating until assignments stop changing.
- Why it works: it is simple, fast, and usually good when clusters are compact and balanced.
- Use when: customer segmentation, image color quantization, or coarse grouping of embeddings.
- Watch out for: sensitivity to initialization, outliers, and the assumption that clusters are roughly spherical.

Hierarchical Clustering

- Core mechanism: merge the closest clusters (agglomerative) or split the whole set step by step (divisive).
- Why it works: it gives a multiscale view of the data, which is useful when you do not know the number of clusters.
- Use when: exploratory analysis and you want a dendrogram or nested grouping.
- Watch out for: cost grows fast with dataset size.

DBSCAN

- Core mechanism: start from a point, expand a cluster if the neighborhood is dense enough, and treat isolated points as noise.
- Why it works: density, not centroid distance, defines a cluster, so it can find arbitrary shapes.
- Use when: geospatial hotspots, anomaly-rich data, and cluster shapes are not spherical.
- Watch out for: choosing epsilon and min_samples, and failure under varying density.

Gaussian Mixture Model

- Core mechanism: model the data as a weighted sum of Gaussian components and fit parameters with Expectation-Maximization.
- Why it works: unlike K-Means, it allows soft assignment and elliptical clusters.
- Use when: probabilistic clustering is useful or cluster covariance matters.

- Watch out for: local optima, singular covariances, and initialization sensitivity.

PCA

- Core mechanism: find orthogonal directions that explain the most variance and project data onto those axes.
- Why it works: correlated dimensions often lie on a lower-dimensional subspace.
- Use when: dimensionality reduction, visualization, and preprocessing before simpler models.
- Watch out for: it is linear and variance is not always the same as usefulness.

t-SNE and UMAP

- Core mechanism: embed data into 2D or 3D while preserving neighborhoods.
- Why it works: high-dimensional semantic structure can become visible in low dimensions.
- Use when: visualization of clusters in embeddings or hidden states.
- Watch out for: the plot is for inspection, not for downstream modeling decisions.

Isolation Forest

- Core mechanism: randomly split features and values; anomalies become isolated in fewer steps than normal points.
- Why it works: rare points are easier to separate from the bulk of the data.
- Use when: fraud spikes, sensor anomalies, account abuse, or any rare event detection.
- Watch out for: it flags unusual points, not necessarily bad points.

4. Scenario-Based Algorithm Selection

Scenario	Best first choice	Strong alternatives	Why this choice
Tabular classification	Logistic Regression or Gradient Boosting	Random Forest, SVM	Tabular data usually rewards strong feature interactions and well-tuned trees.
Tabular regression	Linear Regression baseline, then Gradient Boosting	Random Forest Regressor, MLP	Start simple; move to boosting for non-linear signal.
Text classification	Naive Bayes or Linear SVM	Logistic Regression, small MLP	Sparse bag-of-words features work extremely well for these methods.
Customer segmentation	K-Means	GMM, Hierarchical Clustering	Use centroid methods first unless cluster shape or density demands more.
Geospatial hotspot detection	DBSCAN	HDBSCAN, OPTICS	Density-based methods handle irregular shapes and noise better.
Anomaly detection	Isolation Forest	One-Class SVM, DBSCAN	Anomalies are often rare and not well modeled by supervised labels.
Dimensionality reduction	PCA	UMAP, t-SNE	PCA is the standard first pass; UMAP/t-SNE are stronger for visualization.
Very high-dimensional sparse features	Linear SVM or Logistic Regression	Naive Bayes, SGDClassifier	Linear models scale well and usually beat more complex methods here.

How to choose in practice

- If you need a baseline, start with Logistic Regression, Decision Tree, and Random Forest.
- If the dataset is tabular and accuracy matters, try Gradient Boosting early.
- If the dataset is text and sparse, try Naive Bayes and Linear SVM first.
- If the structure is unlabeled and you need clusters, start with K-Means, then DBSCAN if shape matters.
- If outliers are the focus, use Isolation Forest before more complex methods.
- If you need explainability, favor linear models, shallow trees, or SHAP-ready tree ensembles.

5. Must-Do Practices in ML Projects

- Split before preprocessing that learns statistics from data.
- Use pipelines so training and inference do exactly the same transformations.
- Scale features for distance-based and gradient-based methods; it is usually unnecessary for trees.
- Handle missing values explicitly instead of hoping the model will ignore them.
- Watch for leakage from time, target encoding, aggregation, duplicates, and entity overlap.
- Use validation that matches deployment: time-aware splits for time series, grouped splits for repeated users or sessions.
- Compare against a simple baseline before trusting a complex model.
- Check stability across subgroups, not just aggregate metrics.
- Track model drift after deployment; a good offline score does not guarantee online success.

When to standardize or normalize

Standardization means subtracting the mean and dividing by the standard deviation. Normalization often means scaling to a fixed range, like 0 to 1. The choice depends on the model.

- Use standardization for SVM, k-NN, PCA, logistic regression with regularization, and neural networks.
- Use min-max normalization when the model or feature pipeline benefits from bounded inputs.
- Usually do not scale for tree-based models such as Decision Trees, Random Forests, XGBoost, LightGBM, and CatBoost.
- Fit scaling parameters only on the training split.

6. Quick Cheat Sheet

Question	Good answer	Why
Best first model for tabular data?	Logistic Regression or Gradient Boosting.	Fast baseline first, then stronger non-linear learner.
Best first model for sparse text?	Naive Bayes or Linear SVM.	Sparse high-dimensional features favor linear methods.
Best clustering method for irregular shapes?	DBSCAN.	It uses density, not centroid geometry.
Best dimensionality reduction for preprocessing?	PCA.	Fast and stable linear compression.
Best anomaly detector on tabular data?	Isolation Forest.	Rare points isolate quickly.
Best all-round interpretation-friendly model?	Logistic Regression or shallow trees.	They are easier to explain and debug.

7. Final mental model

Supervised algorithms learn from labels. Unsupervised algorithms discover structure without labels. In real projects, the right choice depends less on hype and more on data shape, scaling, leakage risk, latency, interpretability, and how the model will be used after deployment.

The safest workflow is: start simple, validate correctly, scale preprocessing only on train, compare against baselines, then move to more complex algorithms only when they solve a real failure mode.

End of handbook